OOPSIE: Evaluating QLoRA Adaptation for Schema-Compliant Operational Failure Diagnostics

Onyx Prasai
onyxprasai@gmail.com

November 2026

Abstract

OOPSIE evaluates structured operational-failure diagnostics using a QLoRA adapter paired with an instruction-tuned language model. A frozen four-condition benchmark comprises 40 held-out synthetic failure scenarios and 160 generated responses. Under strict parsing of unmodified outputs, base-model configurations achieved 0/40 schema-conformant responses, while adapted configurations achieved 38/40 with schema guidance and 39/40 without schema text. A post-hoc fence-removal analysis recovered schema-conformant outputs from 40/40 base zero-shot responses and 39/40 base one-shot responses. These findings distinguish direct raw-output compliance from compliance after lightweight normalization and do not establish that adaptation is necessary or preferable to preprocessing. Adapter use also differed alongside input serialization, preventing isolation of the effect of adapter weights. All 160 responses received AI-generated provisional qualitative ratings. The author documented review of 16 flagged responses without changing their ratings; the remaining 144 ratings were adopted without documented case-level review. These exploratory ratings did not establish a diagnostic-quality advantage for adapted configurations. Constrained-decoding baselines were not evaluated. The study motivates controlled comparisons of prompting, normalization, constrained decoding, and adaptation, with schema validation and human oversight required for downstream use.

Keywords: Large Language Models, Parameter-Efficient Fine-Tuning, QLoRA, JSON Schema Conformance, Automated Program Repair, Operational Diagnostics.

1 Introduction

Modern instruction-tuned foundation language models possess extensive parametric knowledge regarding software engineering and debugging, as demonstrated on code generation benchmarks such as HumanEval [CTJ+21] and repository-level issue resolution benchmarks such as SWE-bench [JYW+24]. However, deploying general-purpose chat models as programmatic middleware in automated toolchains presents severe operational challenges. Under standard greedy decoding, unadapted chat models frequently emit conversational pleasantries (*"Certainly, here is the explanation..."*), enclose JSON payloads in Markdown code fences (```` ```json ... ``` ````), or introduce syntactic defects such as illegal escape sequences. When downstream tooling enforces zero-tolerance RFC 8259 JSON validation [Bra17] and rigid schema conformance (JSON Schema Draft 2020-12; [WAHD22, WAH22]), these formatting deviations cause immediate automated ingestion failure.

Prior work has addressed output structuring primarily through decoding-time grammar constraints, such as finite-state automata (FSM) guidance [WL23] or context-free grammar (CFG) token masking [USK+25]. While decoding-time masking guarantees syntax correctness, it imposes non-trivial serving requirements, custom runtime dependencies, and token-level vocabulary masking overhead. An alternative approach is parameter-efficient fine-tuning

(PEFT), such as Low-Rank Adaptation (LoRA; [HSW+22]) and Quantized Low-Rank Adaptation (QLoRA; [DPHZ23]), which internalizes formatting and task conventions directly into the model's weights.

In this work, we present **OOPSIE** (Operational Output Parser & Structured Inference Engine), an open evaluation framework designed to address the following research question:

> *Does QLoRA adaptation improve structured JSON/schema compliance for operational-failure diagnostic reporting, and does it improve qualitative diagnostic reasoning?*

We evaluate this question on a pre-specified, frozen benchmark comprising 40 diverse, held-out software and infrastructure failure scenarios (160 total model completions across four experimental conditions). Under strict parsing of unmodified outputs, adapted configurations achieved 38/40–39/40 schema-conformant responses compared with 0/40 for unadapted base-model configurations ($p \approx 7.28 \times 10^{-12}$). However, a post-hoc analysis shows that base-model failures were overwhelmingly a surface delimiter issue: stripping Markdown code fences recovered 40/40 schema-conformant outputs in zero-shot base inference and 39/40 in one-shot base inference. These results distinguish direct raw-output compliance from compliance after lightweight normalization. A systematic qualitative evaluation found no observable advantage in diagnostic reasoning or technical correctness for adapted conditions.

2 Related Work

2.1 Structured Generation in Foundation Models

Enforcing structured outputs from autoregressive language models is critical for tool use and workflow orchestration. Constrained decoding approaches such as Outlines [WL23] index regular expressions or JSON schemas into deterministic finite automata (DFAs) that mask invalid next-token candidates at each generation step. SynCode [USK+25] generalizes this to context-free grammars (CFGs). In educational and developer error explanation tasks, Leinonen et al. [LHS+23] observed that while LLMs provide helpful diagnostic hints, unadapted models frequently introduce conversational formatting and preamble artifacts that prevent automated programmatic ingestion.

2.2 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA; [HSW+22]) parameterizes weight updates via low-rank decomposition matrices ($W = W_0 + \frac{\alpha}{r}BA$). QLoRA [DPHZ23] combines 4-bit NormalFloat (NF4) base quantization with double quantization and compute-type casting, allowing 12B+ parameter models to be adapted on single commodity GPUs. While PEFT has been evaluated on code generation and general NLP benchmarks [RGG+23,LAZ+23], its efficacy at inducing rigid multi-key JSON schemas under zero-tolerance parsing without grammar masking remains sparsely benchmarked.

2.3 Automated Program Repair and Fault Diagnosis

Automated Program Repair (APR) traditionally encompasses fault localization, patch synthesis, and patch validation against test suites [ZFM+24]. As surveyed by Hou et al. [HZL+24], foundation models excel at synthesizing candidate patches but risk benchmark leakage when evaluated on popular public repositories. OOPSIE is designed not as an autonomous repair agent, but as an advisory operational diagnostic parser that translates unstructured system logs [KS06] into strictly validated machine-readable incident structures.

3 Method

3.1 Canonical Diagnostic Schema

OOPSIE targets a standardized operational diagnostic schema formalized in `data/schema.json` (LF SHA-256: `ce322859f8...`; CRLF SHA-256: `40550bfef0...`) under JSON Schema Draft 2020-12 [WAHD22, WAH22]. Every output must parse as a single JSON object with no additional properties (`additionalProperties: false`). Ten keys are required:

1. `summary`: Non-empty string.
2. `plain_explanation`: Non-empty string.
3. `error_category`: Closed vocabulary enum of 14 values (`dependency`, `configuration`, `environment_variables`, `database`, `authentication`, `authorization`, `network`, `build`, `deployment`, `runtime`, `typescript`, `frontend`, `other`, `unknown`).
4. `likely_causes`: Array of objects with required `description` (non-empty string) and `confidence` (`low`, `medium`, or `high`); no additional properties.
5. `missing_context_questions`: Array of strings.
6. `risk_level`: Enum of three values (`low`, `medium`, `high`).
7. `warnings`: Array of strings.
8. `repair_prompt`: Non-empty string.
9. `verification_steps`: Array of strings.
10. `matched_recipes`: Array of objects with required `id` and `title` (strings) and optional `path`; no additional properties.

Historical Schema Note: Earlier internal design drafts described an alternative schema layout containing fields such as `schema_version`, `root_cause`, and `suggested_fix`. The frozen benchmark and all 160 evaluation passes were executed exclusively against the canonical specification above, stored in `data/schema.json`.

3.2 Training Corpus and Data Governance

The training dataset comprises synthetic scenarios drawn from real-world error patterns across web development, backend databases, cloud infrastructure, container orchestration, and related domains. Each scenario was paired with a 10-key JSON reference payload constructed by the author following the canonical schema. All manifest records carry `source_type: synthetic`. The corpus was partitioned into **85 training examples, 18 validation examples, and 17 test examples** (120 total curated records). All 40 benchmark scenarios were strictly isolated and held out from all training and hyperparameter tuning.

3.3 Model Architecture and QLoRA Adaptation

We configured a QLoRA adapter ($r = 16, \alpha = 32$, dropout 0.05) on top of the public instruction-tuned foundation model `google/gemma-4-12B-it`[1] (11.95B dense parameters; [Gem26,Goo26]). The base model was loaded in 4-bit NormalFloat (NF4) with 16-bit bfloat compute dtype and

[1]Provenance note: The official public Hugging Face repository identifier is `google/gemma-4-12B-it`. Historical training run configuration files (`run_config.json` and `adapter_config.json`) in this repository recorded the lowercase identifier `google/gemma-4-12b-it`; both identifiers resolve to the identical model checkpoint on Hugging Face.

double quantization. Trainable low-rank decomposition matrices were attached across all linear projections in the self-attention and MLP blocks:

$$W = W_0 + \frac{\alpha}{r} BA, \quad r = 16, \quad \alpha = 32 \tag{1}$$

Target modules in the training configuration include attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and feed-forward projections (`gate_proj`, `up_proj`, `down_proj`). Inspection of the saved checkpoint tensor dictionary (`adapter_model.safetensors`) confirms 656 adapter weight tensors totaling 65,568,768 parameters (65.57×10^6) across all 48 transformer layers. Relative to the 11.95B base model, this represents approximately 0.55% of base parameters. Earlier draft claims of 42.6M parameters (0.35%) arose from an incomplete estimate omitting Gemma 4's expanded intermediate dimensions and hybrid attention projection ranks.

The training configuration specified 3 epochs with per-device batch size 1 and 4 gradient accumulation steps (effective batch size 4), AdamW optimization, cosine learning rate scheduling (2×10^{-4} peak), and sequence length 1,536 tokens on an NVIDIA Tesla T4 GPU (14.56 GB allocated VRAM, Kaggle free tier). The evaluation checkpoint corresponds to the saved Step 44 checkpoint (`oopsie-pilot-adapter`) associated with Epoch 2 validation tracking. Execution was managed within an interactive notebook environment without independent training-loss log retention; claims of global checkpoint optimality or exact executed step counts are therefore constrained to the saved configuration and checkpoint artifacts.

4 Experimental Setup

4.1 Four Experimental Conditions

To evaluate the effect of QLoRA adaptation on schema compliance and qualitative diagnostic performance, we tested four conditions across all 40 held-out benchmark scenarios:

- **Condition A (Base Zero-Shot + Schema)**: Unadapted base model with full 710-token JSON schema embedded in the system prompt.
- **Condition B (Adapted + Schema)**: QLoRA adapted model with full JSON schema in the system prompt.
- **Condition C (Base One-Shot + Schema)**: Unadapted base model with full JSON schema and one complete gold-standard input/output demonstration in context.
- **Condition D (Adapted No-Schema)**: QLoRA adapted model with minimal task prompt, completely omitting the schema text.

Critical Methodological Confound: Condition A and Condition B differ in both adapter weights *and* chat-template formatter. Condition A applies the official Gemma instruction-tuned chat template (`apply_chat_template`, `add_generation_prompt=True`); Condition B applies a training-aligned formatter (`add_generation_prompt=False` + `<|turn>model\n` suffix). This comparison therefore evaluates *configured end-to-end systems* under identical prompt text, not an isolated weight-only ablation. The relative contribution of adapter weights versus chat-template format difference cannot be determined from this design. This confound is disclosed explicitly in the frozen evaluation protocol (§9.1 of `OOPSIE-EVAL-PROTOCOL-v1.2.0-frozen`).

4.2 Decoding Parameters and Ingestion Pipeline

All 160 completions were generated using greedy decoding (`temperature = 0.0`, `top_p = 1.0`, `max_new_tokens = 1024`) to ensure deterministic reproducibility. Programmatic evaluation employs a zero-tolerance ingestion pipeline:

1. **Raw JSON Parse**: `json.loads(output)` without Markdown fence stripping or regex extraction.

2. **Lenient JSON Parse**: Extraction after stripping leading/trailing whitespace and Markdown code fences (```` ```json ... ``` ````).

3. **Strict Schema Validation**: Formal JSON Schema Draft 2020-12 validation against `data/schema.json`.

4. **Key Completeness**: Verification that all 10 canonical keys are present and no unexpected keys exist.

4.3 Qualitative Evaluation Protocol and Documented Review Coverage

To evaluate diagnostic reasoning independently of programmatic syntax checks, qualitative ratings were analyzed across five dimensions: Technical Correctness, Root Cause Accuracy, Fix Actionability, Safety Appropriateness, and Repair Prompt Utility.

Documented Review Provenance and Workflow Boundaries:

1. **Initial AI Rating Generation**: All 160 completions across the 40 benchmark scenarios and four experimental conditions were initially rated using an automated AI-assisted evaluation pipeline applying the frozen rubric.

2. **Preparation for Concealed Presentation**: All 160 completions were formatted into eight batch review documents (`batch_01.md` through `batch_08.md`) with randomized condition labels (`System Alpha`, `Beta`, `Gamma`, `Delta`). The first 10 benchmark scenarios (Batches 1–2, comprising 40 completions) were specifically staged for blinded human review.

3. **Documented Author Review**: Rather than de novo blind scoring of all completions, author review evaluated a triage checklist and decision sheet (`author_decision_sheet.md`). This recorded inspection of 16 flagged evaluation entries (category boundaries such as authentication vs. authorization, emulator tests, syntax defects, or workaround flags like `--legacy-peer-deps`).

4. **Overlap and Adoption**: Of the 16 flagged entries subjected to documented author review, exactly 1 overlapped with Batches 1–2 (case `bench-authentication-03`, Beta); the other 15 belonged to Batches 3–8. For all 16 flagged cases, the author retained the provisional rating. The remaining 144 evaluations were adopted into the snapshot manifest without case-level human inspection.

5. **Absence of Inter-Rater Reliability**: No secondary independent human annotator scored the completions; consequently, no Cohen's κ, Krippendorff's α, or inter-rater reliability statistic was calculated.

These qualitative results represent author-adopted AI-assisted ratings and must be interpreted as exploratory observations rather than an independently verified human evaluation.

4.4 Relationship to Constrained Decoding and JSON Mode

This study did not evaluate constrained-decoding baselines, provider JSON modes, or grammar-guided decoding. Therefore, the results do not establish that QLoRA adaptation is necessary or preferable to constrained decoding for achieving schema compliance. Specifically, the following approaches were not evaluated:

Condition	Raw JSON	Lenient JSON	Strict Schema	Key Compl.	Latency (s)	Fail Reason
A: Base Zero-Shot + Schema	0/40 (0.0%)	40/40 (100.0%)	0/40 (0.0%)	0/40 (0.0%)	118.62 ± 12.73	Markdown fences / preambles
B: Adapted + Schema	38/40 (95.0%)	38/40 (95.0%)	38/40 (95.0%)	38/40 (95.0%)	114.42 ± 11.53	Incomplete/syntax defects (2 runs)
C: Base One-Shot + Schema	0/40 (0.0%)	39/40 (97.5%)	0/40 (0.0%)	0/40 (0.0%)	119.35 ± 9.79	Markdown fences / syntax defect
D: Adapted No-Schema	40/40 (100.0%)	40/40 (100.0%)	39/40 (97.5%)	39/40 (97.5%)	99.88 ± 11.93	Missing 1 key (1 run)

Table 1: Programmatic parsing and schema compliance results across 40 held-out scenarios (160 total passes). Lenient JSON reflects parser output after removing Markdown code fences. Latency denotes single sequential execution time on an NVIDIA Tesla T4 GPU (mean ± standard deviation).

- **Grammar-constrained decoding** (e.g., FSM/CFG masking as in Outlines [WL23] or SynCode [USK+25]).
- **Provider JSON modes** (inference-time JSON enforcement offered by hosted model providers).
- **Negative-prompting baselines** (e.g., system-prompt instructions such as *"Output raw JSON only, no markdown, no preamble"*). All conditions shared a base system prompt containing the directive "Return only one JSON object conforming to `data/schema.json`"; however, this directive omitted explicit anti-markdown or anti-preamble instructions. A pure no-directive baseline was therefore not evaluated.

The evaluation scope was restricted to parameter adaptation versus standard prompt-only baselines using identical inference infrastructure, without integration of constrained-decoding runtime engines. Future work should compare QLoRA adaptation against constrained-decoding approaches, JSON-mode decoding, and prompt-only formatting controls to determine whether adaptation is necessary or merely one of several viable paths to structural compliance.

5 Results

5.1 Programmatic Parsing and Schema Compliance

Table 1 reports the quantitative parsing and schema conformance results across all 160 runs.

Unadapted base models (Conditions A and C) achieved 0.0% raw JSON parsing and 0.0% strict schema compliance under the primary zero-tolerance protocol. In all failure cases, the raw model emission was enclosed in Markdown code blocks (```` ```json ... ``` ````) or conversational introductory formatting.

Post-Hoc Lenient Extraction Analysis To evaluate whether base-model non-compliance was exclusively a superficial delimiter framing failure, we parsed and validated outputs after stripping Markdown code fences:

- **Condition A (Base Zero-Shot)**: 40/40 (100.0%) parsed as valid JSON and 40/40 (100.0%) fully satisfied the strict JSON schema, confirming all 10 canonical keys, enum constraints, and type requirements were met once framing delimiters were removed.
- **Condition C (Base One-Shot)**: 39/40 (97.5%) parsed as valid JSON and passed strict schema validation. The single failure (`bench-nextjs-03`) contained an internal syntax defect (a missing opening quote on a key name: `_explanation"`), preventing JSON parsing even after fence stripping.

Correction Notice on Condition C Lenient Parsing: Earlier project summaries and draft tables incorrectly listed Condition C lenient parsing as 40/40 (100.0%). A forensic audit of the frozen benchmark execution records confirmed that the raw completion for `bench-nextjs-03` omitted

the leading quote and identifier for `"plain_explanation"`, causing a `JSONDecodeError` on line 3 column 3 upon parsing. The underlying frozen benchmark results remain unaltered; the corrected count of 39/40 (97.5%) is reported here.

This post-hoc analysis demonstrates that unadapted base models under greedy decoding possessed the latent structural ability to generate schema-compliant diagnostic payloads, but failed the operational pipeline entirely due to enclosing delimiters.

Conversely, QLoRA adaptation eliminated Markdown code fences and preambles from direct emission: Condition B achieved 95.0% (38/40) raw and strict schema compliance, while Condition D achieved 100.0% raw JSON parsing and 97.5% (39/40) strict schema compliance directly out of the model. The two Condition B failures (`bench-firebase-01` and `bench-nextjs-01`) contained invalid backslash escape sequences in their raw JSON output — not framing delimiters — and therefore were not recoverable by fence removal. The single Condition C failure after fence removal (`bench-nextjs-03`) had a missing opening quote on the `plain_explanation` key, causing a `JSONDecodeError` at line 3, column 3. The single Condition D failure (`bench-npm-03`) produced valid JSON but omitted the required `matched_recipes` key while including all other keys. These four cases are the failures *remaining after normalization*, not the only raw-output failures (Conditions A and C each had 40 raw failures, all due to Markdown framing).

5.2 Inferential Statistical Significance

To rigorously test whether the observed improvement in schema compliance is statistically significant, we construct the paired 2×2 contingency table across the 40 scenarios for Condition A vs. Condition B, shown in Table 2.

	Condition B Pass	**Condition B Fail**
Condition A Pass	0 (a)	0 (b)
Condition A Fail	38 (c)	2 (d)

Table 2: Paired 2×2 contingency table for strict schema compliance (Condition A vs. Condition B, $N = 40$ scenarios).

The contingency table yields 38 discordant pairs ($b = 0$, $c = 38$). Under the exact two-sided McNemar test with null hypothesis $H_0 : P(b) = P(c) = 0.5$:

$$p = 2 \times \sum_{k=0}^{0} \binom{38}{k} (0.5)^{38} = 2 \times (0.5)^{38} \approx 7.276 \times 10^{-12} \tag{2}$$

The Edwards continuity-corrected chi-square statistic is:

$$\chi^2_{\text{corrected}} = \frac{(|b - c| - 1)^2}{b + c} = \frac{(|0 - 38| - 1)^2}{38} = \frac{37^2}{38} = \frac{1369}{38} \approx 36.0263 \tag{3}$$

yielding $p \approx 1.947 \times 10^{-9}$ ($df = 1$). The uncorrected McNemar statistic $\chi^2 = 38.00$ yields $p \approx 7.074 \times 10^{-10}$.

Statistical Scope and Limitations: We explicitly specify that this significance claim pertains strictly to the primary outcome of programmatic schema compliance under zero-tolerance greedy parsing on the 40 paired benchmark scenarios. The p-values are unadjusted for multiple hypothesis testing, and the test does not measure, imply, or support any claim of superiority in underlying technical diagnostic correctness.

Dimension	Condition A (Base)	Condition B (Adapted)	Condition C (1-Shot)	Condition D (No-Schema)
Technical Correctness	37 Pass / 3 Partial / 0 Fail	37 Pass / 3 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	37 Pass / 3 Partial / 0 Fail
Root Cause Accuracy	38 Pass / 2 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail
Fix Actionability	38 Pass / 2 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail
Safety Appropriateness	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail
Repair Prompt Utility	39 Pass / 1 Partial / 0 Fail	38 Pass / 2 Partial / 0 Fail	40 Pass / 0 Partial / 0 Fail	37 Pass / 3 Partial / 0 Fail

Table 3: Systematic qualitative evaluation across all 160 completions (40 scenarios × 4 conditions). *Rating provenance: All 160 ratings were generated by an automated AI evaluation pipeline; 40 were prepared for concealed review, and 16 flagged edge cases received documented author review before ratings were adopted without case-level re-annotation. No inter-rater reliability statistic was computed.*

5.3 Inference Latency Profile

Omitting the explicit schema text from the prompt (Condition D) yielded an observed mean single-run latency reduction of 14.54 seconds (−12.7%) compared to Condition B (99.88±11.93 s vs. 114.42 ± 11.53 s). We note that latency was measured during a single sequential run on an isolated NVIDIA Tesla T4 GPU with batch size 1 and unpadded inputs; this latency profile is reported as an operational observation of prompt-length savings rather than a general hardware benchmark guarantee.

5.4 Qualitative Diagnostic Reasoning Review

Table 3 summarizes the qualitative evaluation across all 160 completions.

Qualitative diagnostic ratings were similar across unadapted and adapted models:

- Across all 160 completions, zero completions received a **Fail**, **Severe Risk**, or **Cannot Assess** rating.
- Condition C (Base 1-Shot) was rated 40/40 Pass on technical correctness and root cause accuracy, whereas Conditions B and D were each rated 37/40 Pass and 3/40 Partial.
- In the Partial cases, diagnostic discrepancies (e.g., categorizing an authorization failure as authentication, or misattributing an expired SSL certificate to an intermediate authority) were observed across both base and adapted completions without a clear separation in technical reasoning depth.

6 Discussion

6.1 Raw Direct Emission vs. Superficial Delimiter Framing

A key finding of this investigation is that the 0.0% strict schema compliance observed in unadapted base models under zero-tolerance parsing was primarily a superficial delimiter framing failure. The base model `google/gemma-4-12B-it` already possesses comprehensive parametric knowledge regarding software faults, and the post-hoc normalization analysis demonstrated that 40/40 (100.0%) zero-shot base completions and 39/40 (97.5%) one-shot completions were fully compliant with the canonical schema once outer Markdown code fences were removed.

QLoRA adaptation altered this behavior by eliminating outer Markdown code fences and conversational preambles directly from raw generation, achieving 95.0%–97.5% compliance directly out of the model without post-processing. However, because Condition A and Condition B differ in both adapter weights and chat-template formatting boundaries (the official Gemma template versus the training-aligned `<|turn>model` boundary), this structural change cannot be causally isolated to weight adaptation alone. Furthermore, because removing Markdown code fences achieves 100.0% schema compliance in base zero-shot inference, QLoRA adaptation

cannot be concluded to be necessary or intrinsically superior to simple string post-processing or constrained decoding.

As noted in Section 4.4, constrained-decoding baselines, provider JSON modes, and negative-prompting controls were not evaluated. Future work should directly compare parameter adaptation against constrained-decoding and prompt-only formatting controls.

6.2 Prompt Compaction and Schema Omission

Condition D demonstrates that the adapted model reliably produces the 10-key schema structure even when the schema definition is completely omitted from the prompt, reducing prompt context length by 710 tokens and cutting end-to-end execution latency by 12.7% without degrading schema compliance (97.5%). This indicates that the structural property names and types are reliably reproduced from model parameters.

6.3 Operational Safety Implications

Because parameter adaptation does not enhance underlying diagnostic reasoning, it must not be treated as a substitute for validation. Organizations adopting PEFT models for automated operational pipelines must implement strict validation boundaries:

1. **Syntactic and Schema Gate**: Automatic validation against the formal JSON schema before downstream consumption.
2. **Human-in-the-Loop Gate**: Requiring explicit developer review before executing suggested fixes or repair prompts in production environments.

7 Limitations

We explicitly document the threats to validity and limitations of this study:

1. **Sample Size**: The benchmark contains 40 curated scenarios. While statistically significant for structural parsing ($p < 10^{-11}$), larger corpora are needed to detect subtle qualitative shifts.
2. **Evaluator Scope and Review Coverage**: Initial qualitative rubric ratings across all 160 completions were generated by an automated AI evaluation pipeline. Forty completions (the first 10 scenarios across four conditions) were staged in blinded batches, and 16 flagged edge cases received documented author review via a decision sheet before ratings were adopted without case-level human re-annotation. No Cohen's κ, Krippendorff's α, or multi-rater inter-coder reliability metrics were computed. Furthermore, because base-model completions were framed by Markdown backticks while adapted completions were bare JSON, subtle surface differences may have partially compromised human blinding.
3. **Adapter and Template Confound**: As detailed in Section 4.1, Conditions A and B differ in both adapter weights and chat-template formatting boundary (official Gemma template vs. training-aligned `<|turn>model` boundary). Performance differences reflect the configured systems rather than isolated weight effects.
4. **Synthetic Contexts**: Scenarios were synthetically generated error patterns representing common web and infrastructure failures, evaluated as static isolated logs rather than live interactive environments.
5. **Model and Task Specificity**: Findings are specific to `google/gemma-4-12B-it` and the OOPSIE 10-key schema; generalization to other foundation models or schemas requires empirical verification.

6. **No Constrained-Decoding or JSON-Mode Baselines**: This study did not evaluate grammar-constrained decoding, provider JSON modes, FSM/CFG token masking, or negative-prompting controls. The results do not establish whether QLoRA adaptation is necessary or preferable to these alternatives for achieving schema compliance.

8 Ethics and Safety

Automating system error remediation presents inherent operational risks. Flawed diagnostic suggestions could lead to catastrophic downtime, data loss, or security vulnerabilities (e.g., executing destructive shell commands or disabling security controls). OOPSIE outputs are strictly advisory; all repair prompts and suggested fixes must be validated by human engineers.

9 Reproducibility

All benchmark datasets, evaluation protocols, schemas, and analysis scripts are publicly available in the OOPSIE repository at https://github.com/Onyx-Prasai/OOPSIE. The frozen evaluation bundle and benchmark result archive are published under GitHub Release v1.2.0:

- Evaluation Bundle ZIP SHA-256:
  `259bd473d9d18e0b13b79591ca1e44b0439667131b9e3966468d044ca05e42c5`
- Public Result Archive SHA-256:
  `cf24eec1a5c3347ae826fec4b5cbfb7133a1c4ce92aace3eab2b048bdd37f4eb`

10 Conclusion

This study investigated whether parameter-efficient fine-tuning via QLoRA can improve structural JSON schema compliance for operational software error recovery, and whether it alters underlying diagnostic reasoning. In a pre-specified frozen benchmark across 40 unseen failure scenarios (160 total passes), QLoRA adaptation of `google/gemma-4-12B-it` improved direct raw JSON schema compliance from 0.0% in unadapted base-model conditions to 95.0%–97.5% under zero-tolerance greedy parsing ($p \approx 7.28 \times 10^{-12}$), eliminating conversational preambles and Markdown code fences. Crucially, a post-hoc analysis revealed that unadapted base-model non-compliance was overwhelmingly a surface formatting issue: removing outer Markdown code fences recovered 100.0% (40/40) schema compliance in zero-shot base inference and 97.5% (39/40) in one-shot base inference.

Furthermore, qualitative diagnostic evaluation across all 160 completions did not demonstrate a diagnostic-reasoning or technical-correctness advantage for adapted conditions over the unadapted base model. Within the evaluated configurations, QLoRA adaptation functioned as an effective formatting mechanism for direct raw emission, but adaptation was not causally isolated from chat-template conventions and was not demonstrated to be necessary relative to fence stripping or constrained decoding. Operational software diagnostic pipelines can utilize QLoRA for automated direct JSON formatting, but must maintain rigorous programmatic schema validation gates and mandatory human developer oversight.

AI Use Disclosure

The author used AI-assisted tools during research and manuscript preparation. Initial qualitative ratings for the 160 benchmark completions were generated by an automated evaluation pipeline using a frozen rubric. The author reviewed key outputs, verification findings, and flagged evaluations, and takes full responsibility for the final work. The remaining provisional

qualitative ratings were retained without case-by-case author review and are reported as author-adopted AI-assisted evaluations. These ratings are not independent human inter-rater assessments. AI tools were also used to assist with code development, documentation, manuscript drafting, and consistency checking.

References

[Bra17] Tim Bray. The JavaScript object notation (JSON) data interchange format. RFC 8259, Internet Engineering Task Force (IETF), December 2017.

[CTJ+21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Dennison, Florian Sigler, Nikolas Prashanth, Shyamal Arora, Pranav Chantzis, Mateusz Gray, Kimberly Zwierzchowska, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[DPHZ23] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 10088–10115, 2023.

[Gem26] Gemma Team (Google DeepMind). Gemma 4 technical report. Preprint arXiv:2607.02770, arXiv, July 2026.

[Goo26] Google LLC. google/gemma-4-12b-it model card and repository. Hugging Face, 2026.

[HSW+22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[HZL+24] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2024.

[JYW+24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[KS06] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Special Publication (SP) 800-92, National Institute of Standards and Technology (NIST), September 2006.

[LAZ+23] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Patry, Anton Lozhkov, Mohammad Amini, Loubna Ben Allal, Leandro von Werra, and Harm de Vries. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[LHS+23] Juho Leinonen, Arto Hellas, Sami Sarsa, Brent Reeves, Paul Denny, James Prather, and Brett A. Becker. Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*, volume 1, pages 452–458, 2023.

[RGG+23] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Gabriel Synnaeve, and Louis Martin. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[USK+25] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. SynCode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research (TMLR)*, 2025.

[WAH22] Austin Wright, Henry Andrews, and Ben Hutton. JSON schema validation: A vocabulary for structural validation of JSON. Internet-Draft draft-bhutton-json-schema-validation-01, Internet Engineering Task Force (IETF), June 2022. Draft 2020-12 dialect.

[WAHD22] Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. JSON schema: A media type for describing JSON documents. Internet-Draft draft-bhutton-json-schema-01, Internet Engineering Task Force (IETF), June 2022. Draft 2020-12 dialect.

[WL23] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[ZFM+24] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 33(2):39:1–39:45, 2024.